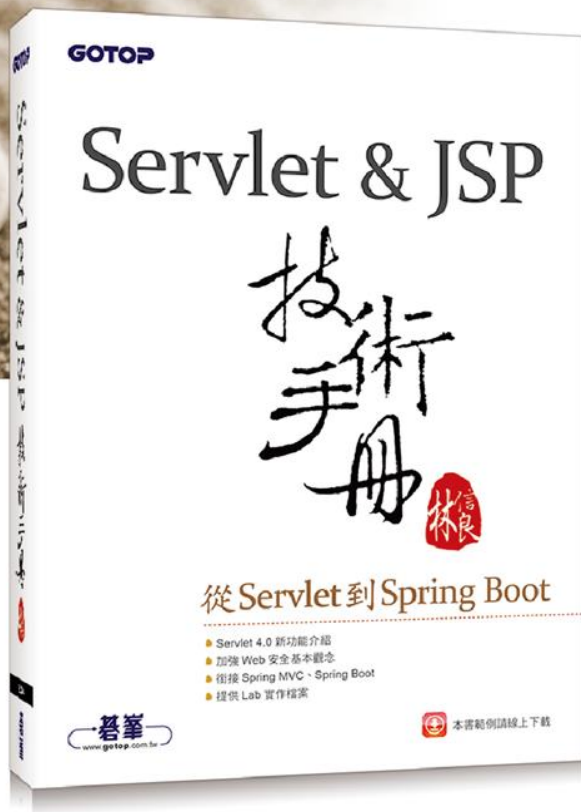




Servlet & JSP

技術手冊

從Servlet到Spring Boot

- Servlet 4.0 新功能介紹
- 加強 Web 安全基本觀念
- 銜接 Spring MVC、Spring Boot
- 提供 Lab 實作檔案

CHAPTER 4

• 會話管理



學習目標

- 了解會話管理基本原理
- 使用Cookie類別
- 使用HttpSession會話管理
- 了解容器會話管理原理

會話管理基本原理

- 每個請求對伺服器來說都是新的訪客請求
- 每次請求時「主動告知」伺服器多次請求間必要的資訊

使用隱藏欄位

①

 $q1=value1\&q2=value2$

②

```
<input type="hidden" name="q1" value="value1">
```

```
<input type="hidden" name="q2" value="value2">
```

③

 $q1=value1\&q2=value2\&q3=value3$

上一頁問卷結果

下一頁問卷結果

```
PrintWriter out = response.getWriter();
out.println("<!DOCTYPE html>");
out.println("<html>");
out.println("<head>");
out.println("<meta charset='UTF-8'>");
out.println("</head>");
out.println("<body>");
```

❶ page 請求參數決定顯示

```
String page = request.getParameter("page"); ← 哪一頁問卷
out.println("<form action='questionnaire' method='post'>");
```

```
if("page1".equals(page)) {
    page1(out);
}
else if("page2".equals(page)) {
    page2(request, out);
}
else if("finish".equals(page)) {
    page3(request, out);
}
out.println("</form>");
out.println("</body>");
out.println("</html>");
```

```
private void page1(PrintWriter out) {
    out.println("問題一:<input type='text' name='p1q1'><br>");
    out.println("問題二:<input type='text' name='p1q2'><br>");
    out.println("<input type='submit' name='page' value='page2'>");
}
```

```
private void page2(HttpServletRequest request, PrintWriter out) {
    String p1q1 = request.getParameter("p1q1");
    String p1q2 = request.getParameter("p1q2");
    out.println("問題三:<input type='text' name='p2q1'><br>");
    out.printf("<input type='hidden' name='p1q1' value='%s'>%n", p1q1);
    out.printf("<input type='hidden' name='p1q2' value='%s'>%n", p1q2);
    out.println("<input type='submit' name='page' value='finish'>");
}
```

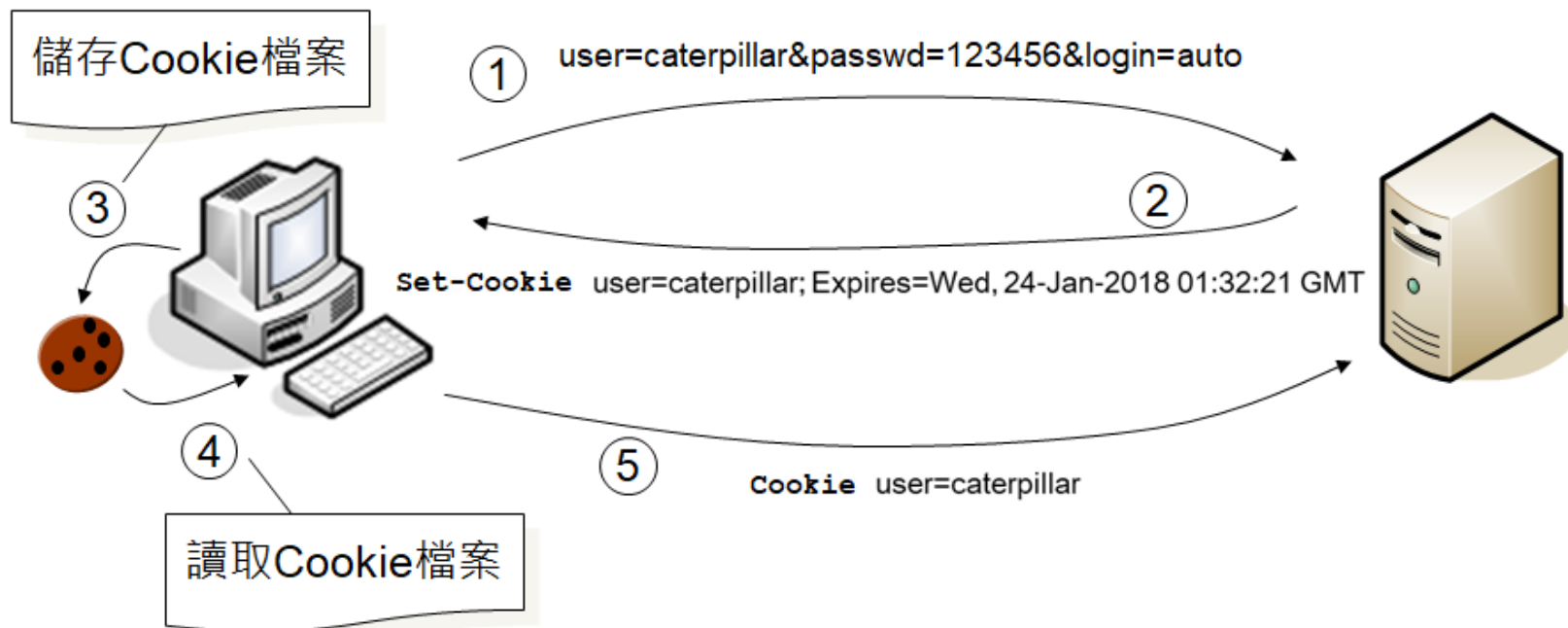
② 第一頁問卷答案，
使用隱藏欄位發送
案，使用隱藏欄位

```
private void page3(HttpServletRequest request, PrintWriter out) {
    out.println(request.getParameter("p1q1") + "<br>");
    out.println(request.getParameter("p1q2") + "<br>");
    out.println(request.getParameter("p2q1") + "<br>");
}
```

使用隱藏欄位

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset='UTF-8'>
  </head>
  <body>
    <form action='questionnaire' method='post'>
      問題三:<input type='text' name='p2q1'><br>
      <input type='hidden' name='p1q1' value='測試一'>
      <input type='hidden' name='p1q2' value='測試二'>
      <input type='submit' name='page' value='finish'>
    </form>
  </body>
</html>
```

使用Cookie



```
Cookie cookie = new Cookie("user", "caterpillar");  
cookie.setMaxAge(7 * 24 * 60 * 60); // 單位是「秒」，所以一星期內有效  
response.addCookie(cookie);
```


使用Cookie

- Cookie的設定是透過set-cookie標頭
- 必須在實際回應瀏覽器之前使用
addCookie() 來新增Cookie實例
- 瀏覽器輸出HTML回應之後再執行
addCookie() 是沒有作用的

使用Cookie

- 可以使用 **setMaxAge()** 設定Cookie的有效期限，設定單位是「秒」
- 預設關閉瀏覽器之後Cookie就失效

取得Cookie

- HttpServletRequest的getCookies()

```
Cookie[] cookies = request.getCookies();
if(cookies != null) {
    for(Cookie cookie : cookies) {
        String name = cookie.getName();
        String value = cookie.getValue();
        ...
    }
}

Optional<Cookie[]> cookies = Optional.ofNullable(request.getCookies());
if(cookies.isPresent()) {
    Stream.of(cookies.get())
        .forEach(cookie -> {
            String name = cookie.getName();
            String value = cookie.getValue();
            ...
        });
}
```

Optional<Cookie> userCookie =

Optional.ofNullable(request.getCookies()) ← ❶ 取得 Cookie
.flatMap(this::userCookie);

if(userCookie.isPresent()) {

Cookie cookie = userCookie.get();

request.setAttribute(cookie.getName(), cookie.getValue());

request.getRequestDispatcher("user.view")
.forward(request, response);

} else {

response.sendRedirect("login.html"); ← ❷ 如果沒有相對應的 Cookie 名稱
與數值，表示尚未允許自動登
入，重新導向至登入表單

private Optional<Cookie> userCookie(Cookie[] cookies) {

return Stream.of(cookies)

.filter(cookie -> check(cookie))

.findFirst();

private boolean check(Cookie cookie) {

return "user".equals(cookie.getName()) && ← ❸ 如果有這個 Cookie 名稱與
"caterpillar".equals(cookie.getValue()); 數值，使用者可以觀看頁面

}

```
protected void doPost(  
    HttpServletRequest request, HttpServletResponse response)  
        throws ServletException, IOException {  
    String name = request.getParameter("name");  
    String passwd = request.getParameter("passwd");  
  
    String page;  
    if("caterpillar".equals(name) && "123456".equals(passwd)) {  
        processCookie(request, response);  
        page = "user";  
    }  
    else {  
        page = "login.html";  
    }  
    response.sendRedirect(page);  
}
```



http://localhost:8080/Session/login.html

名稱:

密碼:

自動登入: ☐

```
private void processCookie(  
    HttpServletRequest request, HttpServletResponse response) {  
    Cookie cookie = new Cookie("user", "caterpillar");  
    if("true".equals(request.getParameter("auto"))) { ← ❶ auto 為 "true"  
        cookie.setMaxAge(7 * 24 * 60 * 60); ← 表示自動登入  
    }  
    response.addCookie(cookie);  
}
```

❷ 設定一星期內有效

取得Cookie

- Cookie若要避免被竊取，可以透過Cookie的 **setSecure()** 設定 **true**，那麼就只會在連線有加密（HTTPS）的情況下傳送Cookie。
- 在Servlet 3.0中，Cookie類別新增了 **setHttpOnly()** 方法
 - 會在set-cookie標頭上附加HttpOnly屬性，在瀏覽器支援的情況下，這個Cookie將不會被客戶端腳本（例如JavaScript）讀取
 - 使用 **isHttpOnly()** 來得知一個Cookie是否被 **setHttpOnly()**

使用URI重寫

①

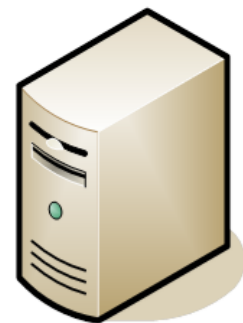
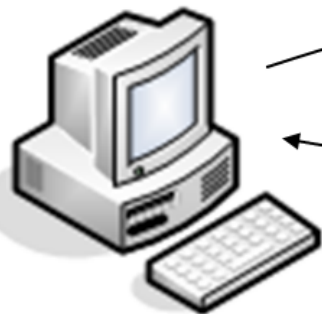
[1](#) [2](#) [3](#) [4](#) [5](#) [6](#) [7](#) [8](#) [9](#) [10](#)

localhost:8080/Session/search?page=2

②

page=2

③



```
<a href=search?page=1>1</a>  
2  
<a href=search?page=3>3</a>  
<a href=search?page=4>4</a>  
...
```

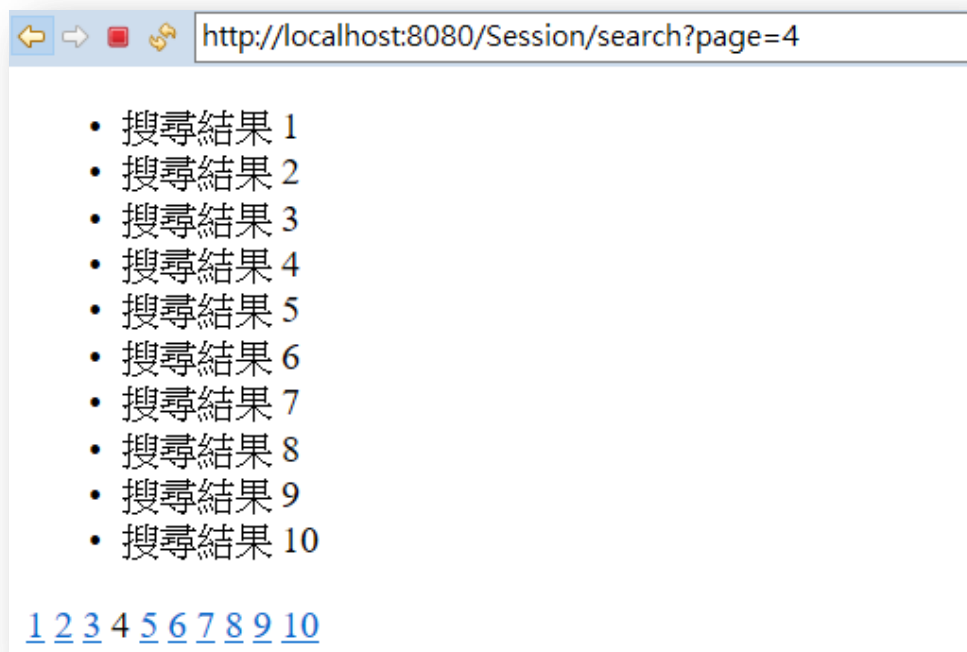
④

[1](#) [2](#) [3](#) [4](#) [5](#) [6](#) [7](#) [8](#) [9](#) [10](#)

localhost:8080/Session/search?page=3

```
int p = Integer.parseInt(page);
IntStream.rangeClosed(1, 10)
    .forEach(i -> {
        if(i == p) {
            out.println(i);
        }
        else {
            out.printf("<a href='search?page=%d'>%d</a>%n", i, i);
        }
    });
```

使用 URI 重寫保留分頁資訊



使用HttpSession

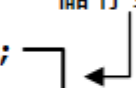
- 用HttpServletRequest的**getSession()**方法取得HttpSession物件

```
HttpSession session = request.getSession();
```

- 會話範圍屬性
 - **setAttribute()**
 - **getAttribute()**

```
private void page2(HttpServletRequest request, PrintWriter out) {  
    String plq1 = request.getParameter("plq1");  
    String plq2 = request.getParameter("plq2");  
    request.getSession().setAttribute("plq1", plq1);  
    request.getSession().setAttribute("plq2", plq2);  
    out.println("問題三:<input type='text' name='p2q1'><br>");  
    out.println("<input type='submit' name='page' value='finish'>");  
}
```

① 改用 HttpSession 儲存第一頁答案



② 改用 HttpSession 取得第一頁答案

```
private void page3(HttpServletRequest request, PrintWriter out) {  
    out.println(request.getSession().getAttribute("plq1") + "<br>");  
    out.println(request.getSession().getAttribute("plq2") + "<br>");  
    out.println(request.getParameter("p2q1") + "<br>");  
}
```



使用HttpSession

- 預設關閉瀏覽器前，取得的HttpSession都是相同的實例
- 直接讓目前的HttpSession失效，可以執行HttpSession的**invalidate()** 方法

使用HttpSession

```
String page;
if("caterpillar".equals(name) && "123456".equals(passwd)) {
    if(request.getSession(false) != null) {
        request.changeSessionId(); ← ❶ 變更 Session ID
    }
    request.getSession().setAttribute("login", name); ← ❷ 設定登入字符
    page = "user";
}
else {
    page = "login.html";
}
response.sendRedirect(page);
```

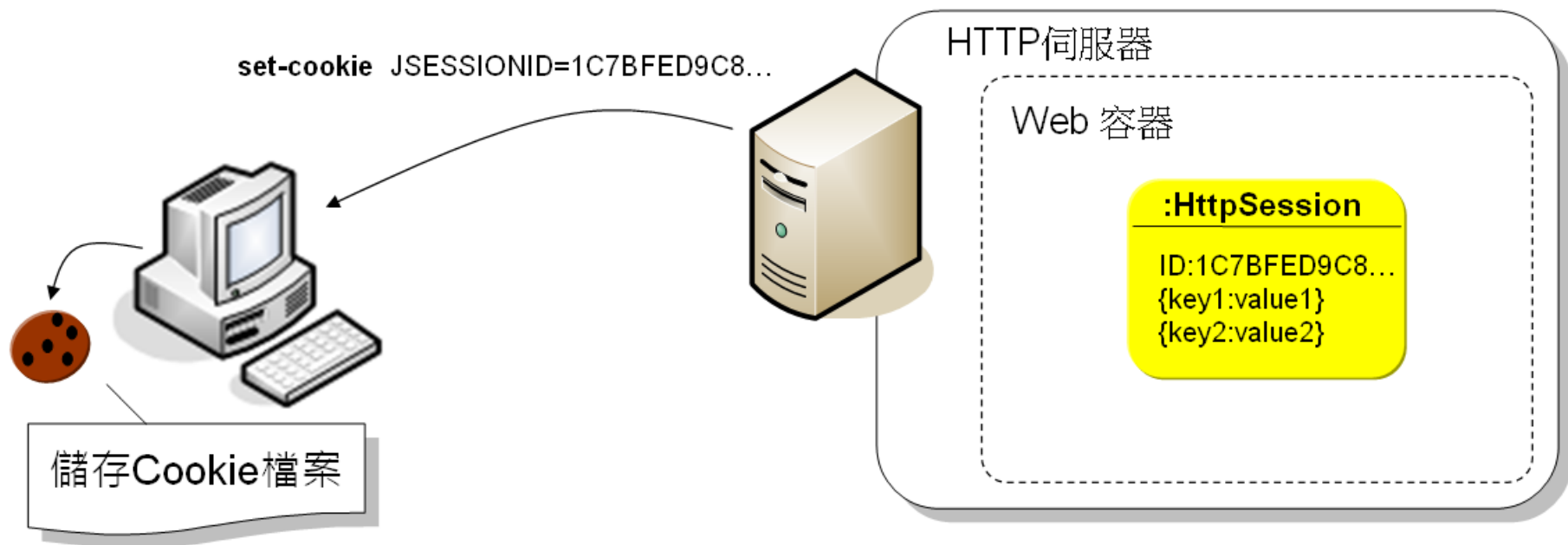
```
protected void doGet(
    HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    HttpSession session = request.getSession();
    Optional<Object> token =
        Optional.ofNullable(session.getAttribute("login"));

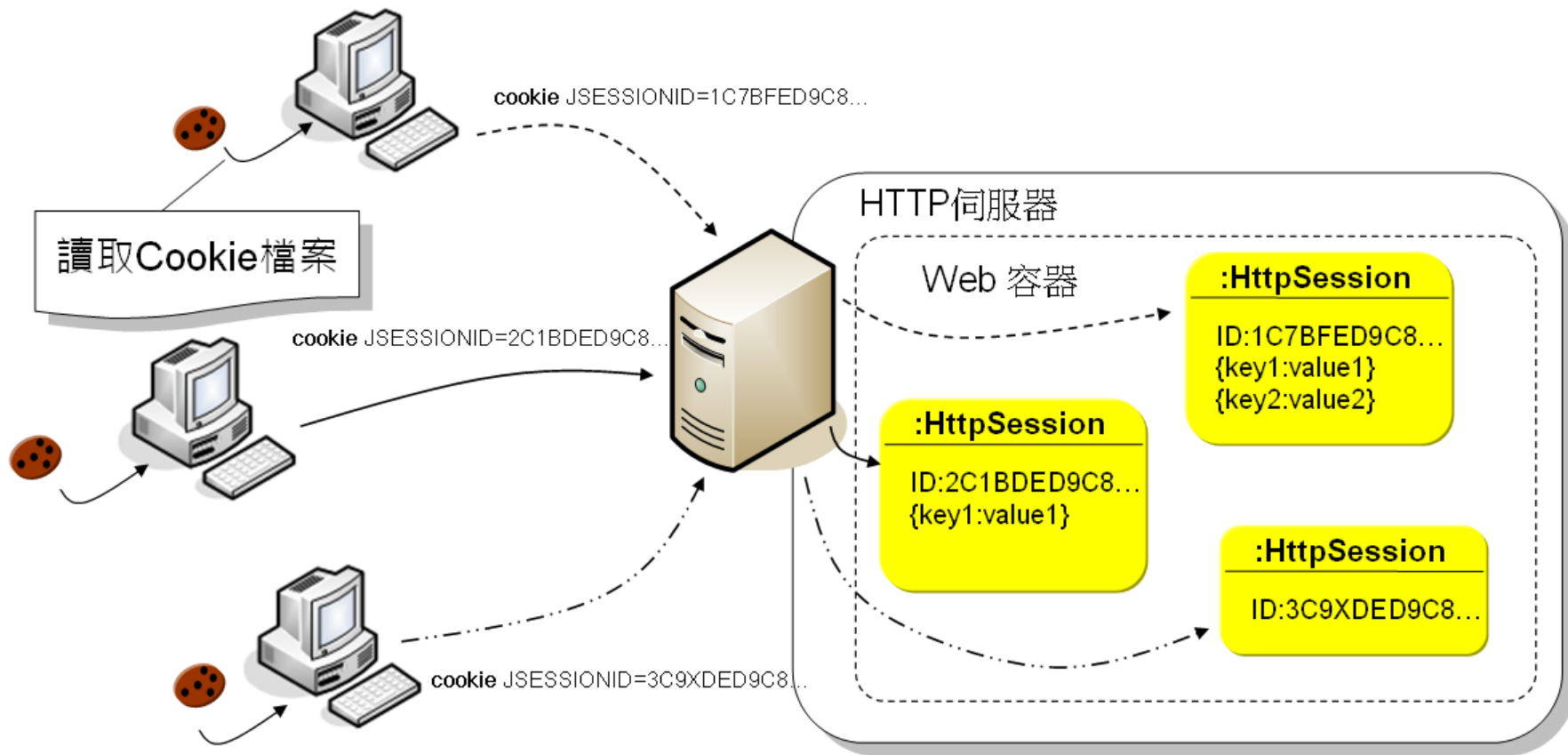
    if(token.isPresent()) {
        request.getRequestDispatcher("user.view") ← ❶ 取得登入字符，轉
            .forward(request, response);          發使用者頁面
    } else {
        response.sendRedirect("login.html"); ← ❷ 無法取得登入字符，重新
    }                                           導向至登入表單
}
```

```
@WebServlet("/logout")
public class Logout extends HttpServlet {
    @Override
    protected void doGet(
        HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        request.getSession().invalidate(); ← 使 HttpSession 失效
        response.sendRedirect("login.html");
    }
}
```

HttpSession會話管理原理



HttpSession 會話管理原理



HttpSession自動失效

- 執行HttpSession的 **setMaxInactiveInterval()** 方法，單位是「秒」
- 在web.xml中設定

```
</web-app ...>  
    略...  
    <session-config>  
        <session-timeout>30</session-timeout>  
    </session-config>  
</web-app>
```

HttpSession自動失效

- HttpSession物件在瀏覽器多久沒活動就失效的時間
- 不是儲存Session ID的Cookie失效時間
- 儲存Session ID的Cookie預設為關閉瀏覽器就失效

SessionCookieConfig

- Servlet 3.0中新增，可在web.xml設定

```
</web-app ...>
...
<session-config>
  <session-timeout>30</session-timeout> <!-- 30 分鐘 -->
  <cookie-config>
    <name>yourJsessionid</name>
    <secure>true</secure>          <!-- 只在加密連線中傳送 -->
    <http-only>true</http-only>    <!-- 不可被 JavaScript 讀取 -->
    <max-age>1800</max-age>        <!-- 1800 秒，不建議 -->
  </cookie-config>
</session-config>
</web-app>
```

HttpSession與URI重寫

- HttpSession預設用Cookie儲存Session ID
- 在使用者禁用Cookie的情況下，仍打算運用HttpSession來進行會話管理，那麼可以搭配URI重寫的
- 可以使用HttpServletResponse的**encodeURL()**協助產生所需的URI重寫

```
Integer count = Optional.ofNullable(  
    request.getSession().getAttribute("count")  
).map(attr -> (Integer) attr + 1)  
    .orElse(0);  
  
request.getSession().setAttribute("count", count);  
  
PrintWriter out = response.getWriter();  
out.println("<!DOCTYPE html>");  
out.println("<html>");  
out.println("<head>");  
out.println("<meta charset='UTF-8'>");  
out.println("</head>");  
out.println("<body>");  
out.println("<h1>Servlet Count " + count + "</h1>");  
out.printf("<a href='%s'>递增</a>%n", response.encodeURL("counter"));  
out.println("</body>");  
out.println("</html>");
```

↑ 使用 encodeURL()

← → ↻ 🏠 📄 localhost:8080/SessionAPI/counter;jsessionid=2CF3BC44ACBFD6B67C9199A6326DB175

Servlet Count 1

递增

HttpSession與URI重寫

- `encodeURL()`
- `encodeRedirectURL()`